
SmartRoom Manager

Fiche technique détaillée — code, fonctions, tests, exécution

RÉALISÉ PAR

Dylan Menga Wanda — dylan.mengawanda@edu.ece.fr

Ngangne Takoudjou Angédarelle — angedarelle.ngangnetakoudjou@edu.ece.fr

Nguadjong Maeva — maevadior.nguadjong@edu.ece.fr

ÉTABLISSEMENT

ECE Paris — B3, cycle ingénieur · Sujet n° 2

Ce document n'est pas un résumé

Il décrit chaque fonction importante, sa signature réelle, ce qu'elle fait et pourquoi elle le fait ainsi. Il liste les 1 481 tests par fichier et par intention. Il donne la procédure d'exécution sous PyCharm, port par port. Lu en entier, il permet d'expliquer le code sans l'avoir devant soi.

Sommaire

1. Ce que représente ce travail

1. Volumétrie mesurée
2. Chronologie des 134 commits

2. Lancer le projet sous PyCharm

1. Les six ports et qui les occupe
2. Interpréteur et dépendances
3. Configuration « API »
4. Configuration « Front »
5. Configuration « Tests »
6. Ordre de démarrage et vérifications

3. La couche domaine — fonctions pures

1. types.py — le vocabulaire
2. availability.py — l'algèbre des intervalles
3. conflicts.py — les six chevauchements
4. rules.py — les dix règles
5. recommendation.py — le score pondéré
6. organisation.py

4. La couche services — ce que l'application fait

1. availability_service — check_slot
2. booking_service — le cycle complet
3. rules_service, auth_service, mail_service, audit_service

5. Le modèle de données

1. Les 46 entités
2. La contrainte EXCLUDE, ligne par ligne
3. Les autres garanties

6. L'API — 160 opérations

7. Le front — hooks, adaptateurs, écrans

8. Les tests — 1 481, fichier par fichier

1. Domaine
2. Services et API
3. Assistant
4. Interface
5. Parcours Playwright

9. Les défauts trouvés et corrigés

10. Questions du jury — réponses argumentées

1. Ce que représente ce travail

Un long travail, et de quoi le mesurer.

Un long travail

Ce projet n'est pas une maquette rendue en une semaine. Il représente **134 commits sur treize jours, 61 592 lignes** de code hors tests, **1 481 tests** écrits en parallèle du code, et une campagne de vérification finale écran par écran qui a produit à elle seule une dizaine de corrections.

L'ampleur dépasse le cadre du sujet, qui demandait un portail avec calendrier, recherche, recommandation et statistiques. Nous avons livré cela, et nous y avons ajouté ce que le problème imposait : une garantie d'unicité au niveau de la base, un moteur de règles configurable, un journal d'audit opposable, et un espace d'administration complet.

1.1 Volumétrie mesurée

Élément	Quantité	Comment c'est mesuré
Lignes de code hors tests	61 592	<code>wc -l</code> sur <code>src/</code> et <code>app/</code>
Commits	134	<code>git log --oneline wc -l</code>
Tables PostgreSQL	48	<code>information_schema.tables</code>
Contraintes CHECK et EXCLUDE	110	<code>pg_constraint</code>
Index (dont partiels)	161 (51)	<code>pg_indexes</code>
Types énumérés	18	<code>pg_type</code> <code>typtype='e'</code>
Migrations Alembic	10	<code>alembic/versions/</code>
Opérations d'API	160 sur 128 chemins	<code>/openapi.json</code>
Entités SQLAlchemy	46	<code>app/models/*.py</code>
Services métier	16	<code>app/services/</code>
Écrans React	52 (33 + 19)	<code>src/pages/</code>
Composants React	152	<code>src/components/</code>
Hooks personnalisés	17	<code>src/hooks/</code>
Tests API	1 026 en 37 fichiers	<code>pytest</code>
Tests interface	455 en 41 fichiers	<code>vitest</code>
Parcours Playwright	16 en 3 fichiers	<code>e2e/</code>

1.2 Chronologie

Date	Ce qui a été produit
20 août	Espace utilisateur complet côté interface, centre d'aide, page d'accueil.

22 août	Espace d'administration : seize écrans branchés sur les mêmes moteurs.
23 août	Modèle PostgreSQL et socle FastAPI (phase 2). Moteur de disponibilité et API de réservation (phase 3). Moteur de recommandation (phase 4).
24 août	Refonte : domaine métier pur, API v1 unique, ancienne couche retirée.
25–31 août	Règles par portée, récurrence, support, audit, notifications, assistant, statistiques. Déploiement Docker, Caddy, sauvegardes, intégration continue.
1 ^{er} sept.	Connexion Google, mentions légales, envoi réel des courriels, campagne de vérification écran par écran.

2. Lancer le projet sous PyCharm

Six ports, trois configurations d'exécution, un ordre de démarrage.

2.1 Les six ports et qui les occupe

Port	Service	Rôle et vérification
55441	PostgreSQL 16 + pgvector	Conteneur <code>smartroom-pg-dev</code> . Le port 55441 est celui de l'hôte ; à l'intérieur du conteneur, PostgreSQL écoute toujours 5432. Ce décalage évite d'entrer en conflit avec une autre base installée sur le poste.
8000	API FastAPI	Documentation interactive sur <code>http://127.0.0.1:8000/docs</code> , schéma sur <code>/openapi.json</code> .
5180	Interface Vite	Fixé dans <code>vite.config.js</code> avec <code>strictPort: true</code> : si le port est pris, Vite refuse de démarrer au lieu d'en choisir un autre en silence.
1025	SMTP local	Relais de courrier de développement : rien ne sort de la machine.
8025	Boîte de réception locale	Interface web pour relire les courriels envoyés.
11434	Ollama	Modèle local de l'assistant. Facultatif : l'application démarre sans.

Le piège du port 8000

Sous Windows, un processus `uvicorn` tué peut laisser des processus enfants qui gardent la socket ouverte. `netstat` montre alors le port occupé par un PID qui n'existe plus. Avant de relancer l'API, vérifier :

```
Get-CimInstance Win32_Process -Filter "Name like '%python%'" |  
Where-Object { $_.CommandLine -match 'uvicorn|spawn_main' } |  
ForEach-Object { Stop-Process -Id $_.ProcessId -Force }
```

2.2 Interpréteur et dépendances

L'environnement virtuel est à la racine du dépôt, pas dans `smartroom-api/`. Dans PyCharm :

1. **File** → **Settings** → **Project** → **Python Interpreter**
2. **Add Interpreter** → **Existing environment**
3. Choisir `<projet>/venv/Scripts/python.exe`

Installation des dépendances, une seule fois :

```
.venv\Scripts\python -m pip install -r smartroom-api\requirements.txt
```

2.3 Configuration d'exécution « API »

Run → **Edit Configurations** → **+** → **Python**

Champ	Valeur
Name	API
Run	module → uvicorn
Parameters	app.main:app --reload --host 127.0.0.1 --port 8000

Working directory	<projet>\smartroom-api
Python interpreter	<projet>\.venv\Scripts\python.exe

Deux points qui font échouer le démarrage

Le répertoire de travail. Il doit être `smartroom-api` et non la racine : `app.main` ne se résout pas autrement, et `.env` est lu relativement à ce répertoire.

L'hôte. Écrire `127.0.0.1` et non `localhost`. Sur Windows, `localhost` se résout d'abord en IPv6 (`::1`) ; si le conteneur ou le proxy n'écoute qu'en IPv4, la connexion échoue sans message clair.

2.4 Configuration d'exécution « Front »

+ → **npm**

Champ	Valeur
Name	Front
package.json	<projet>\smartroom-front\package.json
Command	run
Scripts	dev

Vite sert l'interface sur `5180` et relaie `/api` et `/media` vers `http://127.0.0.1:8000`. Le relais est déclaré dans `vite.config.js` ; la cible est réglable par la variable `VITE_API_TARGET` si le port 8000 est déjà pris.

2.5 Configuration d'exécution « Tests »

Champ	Valeur
Type	Python tests → pytest
Target	Custom → smartroom-api\tests
Working directory	<projet>\smartroom-api

Les tests d'intégration exigent que la base tourne : ils créent et détruisent un schéma à chaque session. Côté interface, `npm test` lance Vitest, qui n'a besoin d'aucun service.

2.6 Ordre de démarrage

1. `docker compose up -d db courriel` base 55441, courrier 1025/8025
2. `cd smartroom-api && alembic upgrade head`
3. `python -m scripts.seed` jeu de démonstration
4. Run « API » → `127.0.0.1:8000/docs`
5. Run « Front » → `localhost:5180`

Vérifications rapides, dans cet ordre :

Commande	Réponse attendue
<code>docker ps</code>	<code>smartroom-pg-dev</code> et <code>smartroom-courriel</code> présents
<code>curl 127.0.0.1:8000/openapi.json</code>	200, un JSON de plusieurs centaines de kilo-octets
<code>curl localhost:5180</code>	200, la page HTML de l'application

Comptes de démonstration : le script de peuplement crée des comptes utilisateur et cinq comptes d'administration aux périmètres différents, tous avec le mot de passe `smartroom2026`. L'administration se rejoint par `/admin/connexion`.

3. La couche domaine — fonctions pures

Six modules qui ne connaissent ni base de données ni HTTP. C'est ici que vivent les règles.

Pourquoi cette couche existe

Les fonctions de `app/domain/` prennent des valeurs et rendent des valeurs. Aucune n'ouvre de session, aucune ne connaît FastAPI. Conséquence directe : « ce créneau respecte-t-il le préavis ? » se teste sans base, en quelques millisecondes, et se relit sans dérouler une requête SQL.

C'est ce qui a permis, en fin de projet, de corriger des règles sans rien casser ailleurs : on modifie une fonction pure et son test, pas une route et son mock.

3.1 types.py — le vocabulaire commun

Ce module ne contient aucune logique : il définit les objets que toutes les autres fonctions échangent. Tous sont `frozen=True, slots=True` — immuables et compacts.

Type	Ce qu'il représente
TimeSlot	Un intervalle fermé-ouvert <code>[début, fin)</code> en UTC, avec <code>intersection</code> , <code>gap_to</code> , <code>expanded</code> , <code>duration</code> .
OverlapKind	Les sept qualifications géométriques : identique, englobant, englobé, partiel début, partiel fin, adjacent, aucun.
RuleCode	Les dix codes de violation : durée min/max, horizon min/max, passé, quota, hors ouverture, fermeture, capacité, battement.
RuleSet	Les neuf règles résolues pour une salle donnée.
BookingRef	Une réservation existante, réduite à ce dont le domaine a besoin.
Conflict	Un chevauchement qualifié : type, durée recoupée, écart.
RoomProfile · UserProfile · SearchCriteria	Les entrées du moteur de recommandation.
Score · ScoreComponent · ScoredRoom	Le résultat du score, décomposé par critère.
Alternative	Une proposition de repli après conflit.

Les valeurs par défaut du sujet

RuleSet.defaults()			
min_duration	30 minutes		
max_duration	4 heures		
buffer	15 minutes	battement entre deux réunions	
min_advance	15 minutes	préavis minimal	
max_advance	60 jours	horizon maximal	
max_active_bookings	10		
cancel_deadline	1 heure		
checkin_window	10 minutes		
validation_capacity_threshold	20 places		

Ces valeurs ne servent que si la table `booking_rules` est vide. En fonctionnement normal, elles viennent de la base et sont modifiables depuis l'administration.

3.2 availability.py — l'algèbre des intervalles

Neuf fonctions, toutes sur des intervalles triés. C'est le socle du calcul de disponibilité.

```
merge(slots: Iterable[TimeSlot]) -> tuple[TimeSlot, ...]
```

Fusionne les intervalles qui se touchent ou se chevauchent. Trie d'abord par début, puis parcourt une seule fois en étendant l'intervalle courant tant que le suivant le rejoint.

Pourquoi : sans fusion préalable, soustraire dix réservations d'une journée demanderait dix passes ; après fusion, une seule suffit.

```
subtract(base: TimeSlot, busy: Sequence[TimeSlot]) -> tuple[TimeSlot, ...]
```

Retire les intervalles occupés d'un intervalle de base et rend les trous restants. C'est l'opération centrale : « la salle ouvre de 8 h à 20 h, voici ce qui est pris, que reste-t-il ? »

```
free_slots(...) -> tuple[TimeSlot, ...]
```

Compose les précédentes : fenêtres d'ouverture, moins les fermetures, moins les réservations, moins le battement. Le résultat est *réserveable tel quel*, pas « probablement libre ».

```
open_windows(...) · daily_windows(...)
```

Traduisent des horaires d'ouverture exprimés en heure locale (« du lundi au vendredi, 8 h – 20 h ») en intervalles UTC pour une période donnée.

Le piège du changement d'heure : la conversion se fait *jour par jour* avec `ZoneInfo`, jamais par un décalage constant. Le dernier dimanche d'octobre compte 25 heures ; un décalage figé décalerait toutes les fenêtres d'une heure ce jour-là.

```
is_free(slot, busy, *, buffer) -> bool · is_within(slot, windows) -> bool
```

Prédicats de commodité pour les appelants qui n'ont besoin que d'un oui ou d'un non.

3.3 conflicts.py — les six chevauchements

```
classify(candidate: TimeSlot, existing: TimeSlot) -> OverlapKind
```

Qualifie géométriquement deux intervalles. Les prédicats sont ordonnés *du plus spécifique au plus général* : l'identité est un cas particulier d'englobement et doit être reconnue avant lui.

```
if existing.start == candidate.start and existing.end == candidate.end:
    return IDENTIQUE
if existing.start <= candidate.start and existing.end >= candidate.end:
    return ENGLOBANT
if existing.start >= candidate.start and existing.end <= candidate.end:
    return ENGLOBE
if existing.start < candidate.start < existing.end < candidate.end:
    return PARTIEL_DEBUT
if candidate.start < existing.start < candidate.end < existing.end:
    return PARTIEL_FIN
if existing.end == candidate.start or candidate.end == existing.start:
    return ADJACENT
return AUCUN
```

Le cas adjacent est le piège classique. 10 h – 11 h et 11 h – 12 h ne se chevauchent pas : la borne haute est exclue. La fonction le reconnaît explicitement plutôt que de le confondre avec un recouvrement, et `RECOUVREMENTS` — l'ensemble des cinq vrais recouvrements — ne le contient pas.

```
qualify(candidate, existing: BookingRef) -> Conflict
```

Enrichit la qualification : type, durée effectivement recoupée, et écart quand il n'y a pas recouvrement. C'est ce qui permet d'afficher « chevauche de 30 minutes » plutôt qu'un simple refus.

```
detect(candidate, existing, *, buffer) -> tuple[Conflict, ...]
```

Applique `qualify` à toutes les réservations voisines et retient celles qui bloquent, battement compris.

```
propose_alternatives(...) -> tuple[Alternative, ...]
```

Construit les propositions de repli après un conflit : même salle à un autre créneau, autre salle au même créneau, ou créneau proche. Chaque alternative porte son écart au souhait initial, ce qui permet de les classer.

Pourquoi : un refus sans issue renvoie l'utilisateur à zéro. C'est précisément ce que le sujet reproche à la réservation manuelle.

```
arbitration_brief(...) -> ClaimantFile · seniority(conflicts) -> BookingRef | None
```

Préparent l'arbitrage d'un administrateur : qui a demandé quoi, depuis quand, avec quelles justifications. `seniority` désigne la réservation la plus ancienne, critère par défaut quand rien ne départage.

3.4 rules.py — les dix règles

Chaque règle est une fonction indépendante qui rend un tuple de violations. Aucune ne lève d'exception : elles *rapportent*, et l'appelant décide.

Fonction	Ce qu'elle vérifie
<code>check_duration(slot, rules)</code>	Durée entre <code>min_duration</code> et <code>max_duration</code> .
<code>check_horizon(slot, now, rules)</code>	Préavis minimal, horizon maximal, et créneau déjà passé.

<code>check_quota(active, rules)</code>	Nombre de réservations actives contre <code>max_active_bookings</code> .
<code>check_capacity(attendees, capacity)</code>	Participants contre capacité de la salle.
<code>check_opening(slot, windows)</code>	Créneau entièrement contenu dans les heures d'ouverture.
<code>check_closure(slot, closures)</code>	Aucune fermeture exceptionnelle sur la période.
<code>check_buffer(slot, neighbours, rules)</code>	Battement respecté avec les réservations voisines.
<code>evaluate(...)</code>	Compose les sept précédentes et rend l'ensemble des violations.
<code>requires_validation(attendees, rules)</code>	Au-delà du seuil, une validation humaine est requise.
<code>can_cancel(slot, now, rules)</code>	Délai d'annulation non dépassé.
<code>is_releasable(...)</code>	Fenêtre de check-in écoulée sans validation : la salle peut être rendue.

3.5 recommendation.py — le score pondéré

Six composantes, pondérées, dont la somme **doit** faire 100 — la classe le vérifie à la construction et lève sinon.

```
class Weights:
    capacity = 30      adéquation à l'effectif
    equipment = 25     équipements demandés présents
    building = 15      bâtiment préféré
    occupancy = 12     salle peu sollicitée
    floor = 10         étage préféré
    history = 8        salles déjà réservées par le compte

    def __post_init__(self):
        if sum(...) != 100:
            raise ValueError(...)
```

Fonction	Comportement
<code>capacity_fit</code>	Sous-capacité éliminatoire , surdimensionnement seulement pénalisé. Une salle de 4 places pour 10 personnes n'est pas mal notée : elle est écartée.
<code>equipment_fit</code>	Proportion des équipements demandés effectivement présents.
<code>building_fit · floor_fit</code>	Correspondance avec les préférences du compte.
<code>occupancy_fit</code>	Favorise les salles peu sollicitées, pour répartir l'usage.
<code>history_fit</code>	Favorise les salles que le compte a déjà réservées.
<code>score_room(...)</code>	Compose les six et rend un <code>Score</code> décomposé.
<code>eligibility_issues · is_eligible</code>	Séparent l' éligibilité du classement .
<code>justify(score, blocker)</code>	Génère la phrase d'explication à partir des composantes.

Le score ordonne, il n'élimine pas

Une salle qui ne convient pas n'est pas mal notée : elle n'est pas proposée du tout, et cela relève de `is_eligible`, de la disponibilité et des règles. Le score n'intervient qu'ensuite, pour ordonner ce qui reste.

La justification est **générée depuis les composantes**, jamais rédigée à la main. L'écran n'affiche pas « score 87 » mais « capacité adaptée, vidéoprojecteur présent, même bâtiment que votre dernière réservation ». Un classement qu'on ne peut pas expliquer n'inspire pas confiance.

3.6 organisation.py

```
est_externe(email: str | None) -> bool
```

Rend vrai si le domaine de l'adresse ne figure pas dans `ORGANISATION_DOMAINS`. Sans liste configurée, **personne** n'est externe : mieux vaut ne rien signaler que marquer tout le monde — une étiquette portée par chaque ligne ne distingue plus rien.

4. La couche services

Seize services orchestrent le domaine, la base et les effets de bord.

4.1 availability_service — la fonction pivot

```
check_slot(session, *, room_id, slot, attendees=1, requester_id=None, ignore_booking_id=None, now=None,
check_quotas=True) -> SlotReport
```

Rôle : réservabilité d'un créneau précis, règles violées énumérées. C'est la fonction que traverse toute création ou modification.

Le verdict ne décide rien : il rapporte. L'appelant choisit de forcer une règle — et jamais un chevauchement, que la base refuserait de toute façon.

Déroulé :

1. charger_salle(session, room_id)	la salle et son étage
2. load_rules(session, salle)	cascade salle → bâtiment → global
3. local_days(slot, FUSEAU)	jours locaux couverts
4. open_windows_for(...)	fenêtres d'ouverture
5. load_closures(...)	fermetures exceptionnelles
6. load_bookings(..., slot.expanded(buffer))	voisines, battement compris
7. conflicts.detect(slot, voisines, buffer)	domaine pur
8. count_active_bookings(...)	pour le quota
9. rules.evaluate(...)	domaine pur
→ SlotReport(conflits, violations, ...)	

À retenir pour l'oral : les étapes 1 à 6 et 8 touchent la base ; les étapes 7 et 9 sont des fonctions pures. La frontière est nette, et c'est exactement ce qui rend les cas limites testables sans base.

```
load_rules(session, salle) -> RuleSet
```

Résout la cascade salle → bâtiment → global. Chaque niveau ne surcharge que ce qu'il déclare ; la fonction `_specificite` ordonne les candidats du plus précis au plus général en SQL, pour ne rendre qu'une ligne.

4.2 booking_service — le cycle de vie complet

```
create_booking(session, *, room_id, owner_id, slot, title, attendees, participants=(), source,
ignore_rules=False, now=None) -> tuple[Booking, IssuedCode | None]
```

Déroulé : vérifie le propriétaire, appelle `check_slot`, applique le verdict, insère la réservation, émet le code d'accès si la salle exige un badge, invite les participants, journalise et prévient par courriel.

Ne valide pas la transaction. Le `commit` appartient à l'appelant, qui seul sait si d'autres écritures l'accompagnent.

```
_traduire_course(erreur: IntegrityError) -> None
```

La fonction la plus importante du service. Elle transforme une violation de la contrainte `EXCLUDE` en conflit lisible :

```
sqlstate = getattr(getattr(erreur, "orig", None), "sqlstate", None)
if sqlstate == EXCLUSION_VIOLATION:          SQLSTATE 23P01
    raise SlotConflictError(
        "Ce créneau vient d'être réservé par quelqu'un d'autre. "
        "Rafraîchissez la page pour voir les disponibilités à jour."
    ) from erreur
raise erreur                                toute autre violation remonte telle quelle
```

Pourquoi ce `raise erreur final` : toute autre violation d'intégrité est un défaut de code. La déguiser en erreur métier masquerait un bug derrière un message rassurant.

C'est ici que se joue la concurrence. Deux requêtes simultanées passent toutes deux `check_slot` — les deux voient le créneau libre. Une seule survit à l'écriture ; l'autre reçoit ce message.

```
issue_access_code(session, reservation, *, now=None) -> IssuedCode | None
```

Émet un code temporaire si la salle exige un badge. Forme `E-6477` : une lettre tirée du code du bâtiment, un tiret, quatre chiffres tirés par `secrets.randbelow`.

Le clair est rendu à l'appelant et jamais persisté : la base ne garde que `CRYPT.hash(clair)` et un indice `E-****`.

Le paramètre `now` suit celui de l'appelant. Lire l'horloge système ici produirait un `issued_at` postérieur à `expires_at` dès qu'une réservation est datée autrement que « maintenant » — ce que la contrainte `ck_booking_access_codes_expiry` refuse.

```
check_in(session, booking_id, *, code, now=None) -> Booking
```

Valide la présence, code d'accès à l'appui. La fenêtre est bornée des deux côtés : avant le début il n'y a rien à valider, après `checkin_window` le créneau est réputé libérable.

```
if now < creneau.start:                → « La validation ouvre au début du créneau. »
if now >= creneau.start + checkin_window: → « Fenêtre de validation dépassée. »
if salle.badge_required and not CRYPT.verify(code, actif.code_hash):
    → « Code d'accès incorrect. »
```

Le code est vérifié tel qu'il a été émis, tiret compris. Normaliser des deux côtés reviendrait à accepter une chaîne que le serveur n'a jamais produite, donc à élargir en silence ce qui ouvre une porte.

```
mark_late(session, booking_id, *, delai_min=None, now=None) -> Booking
```

Signale un retard : le créneau reste réservé au-delà de la fenêtre. Sans cela, la tâche de libération rendrait la salle à quelqu'un qui arrive avec dix minutes de retard.

La marque vaut validation de présence : elle pose `checked_in_at`. La durée annoncée est facultative, ne décale aucune règle, et part au journal de la réservation.

```
cancel_booking(session, booking_id, *, reason, actor_id=None, now=None) -> Booking
```

Annule, motif obligatoire. Une annulation tardive n'est pas refusée — le créneau doit être libéré même au dernier moment — mais elle est signalée dans la frise.

Une présence validée ferme la porte : on ne défait pas une occupation constatée. La base le garantit par `ck_bookings_cancelled_not_checked_in` ; le service le vérifie avant, pour rendre un 422 nommé plutôt qu'un 500.

```
release_no_shows(session, now=None) -> list[Booking] · close_finished_bookings(session, now=None) -> int
```

Tâches planifiées, exécutées toutes les 300 secondes. La première libère les créneaux commencés dont la présence n'a jamais été validée ; la seconde clôt les réservations terminées. Ni l'une ni l'autre ne touche aux blocages administratifs.

```
update_booking · add_participant · remove_participant · respond_to_invitation · list_participants · reissue_access_code · create_blocking
```

Le reste du cycle : modification avec nouveau code d'accès, gestion des invités, réponse à une invitation par jeton signé, réémission de code après perte, et blocage administratif d'une salle.

4.3 Les autres services

Service	Responsabilité
rules_service	Lecture et écriture des règles par portée, prévisualisation de l'effet d'une règle sur 30 jours (<code>preview_rule</code>), heures d'ouverture et fermetures.
auth_service	Connexion, rotation des jetons de rafraîchissement, réinitialisation de mot de passe, connexion Google (<code>login_google</code>).
google_service	Vérification du jeton d'identité : signature RS256 contre le JWKS de Google, émetteur, destinataire , adresse confirmée.
mail_service	Rendu des gabarits, notification applicative, file d'envoi différée après le COMMIT.
audit_service	Écriture du journal : qui, quoi, quand, depuis quelle adresse, et le diff avant/après.
parc_service	Bâtiments, étages, salles, équipements, plans, photos. <code>list_rooms</code> porte les filtres.
recommendation_service	Charge les profils de salles et appelle le domaine pour classer.
recurrence_service	Engendre les occurrences d'une réservation répétée, indépendantes les unes des autres.
support_service · access_request_service	Tickets, base de connaissances, demandes d'accès exceptionnel.
stats_service	Occupation, absences, rapports. S'appuie sur la vue matérialisée <code>mv_room_occupancy_hourly</code> .
notification_service · users_service · chat_service	Notifications, comptes et préférences, assistant.

Pourquoi les courriels partent après le COMMIT

Les envois sont différés hors de la transaction. Expédier avant le `COMMIT` annoncerait une réservation qu'un `ROLLBACK` ferait disparaître : le destinataire recevrait la confirmation d'une réunion qui n'existe pas. La notification applicative, elle, est écrite *dans* la même transaction — un `ROLLBACK` emporte les deux ensemble.

5. Le modèle de données

46 entités, 110 contraintes, et une garantie qui ne se contourne pas.

5.1 Les 46 entités

Domaine	Entités
Identité	User · UserPreference · RefreshToken · PasswordResetToken
Administration	AdminAccount · PermissionGroup · Permission · AdminPermission · AdminInvitation · AdminInvitationPermission
Parc	Building · Floor · FloorPlan · Room · RoomPlacement · Equipment · RoomEquipment · RoomPhoto
Réservation	Booking · BookingParticipant · BookingEvent · BookingAccessCode · RecurrenceRule
Règles	BookingRule · OpeningHour · ClosurePeriod · ClosureBuilding · ClosureRoom
Support	Ticket · TicketMessage · TicketResponseTemplate · AccessRequest
Connaissance	FaqCategory · FaqArticle · FaqFragment · FaqArticleLink · ChatbotIntent · ChatbotIntentKeyword
Assistant	ChatRole · ChatConversation · ChatMessage · ChatTour
Communication	Notification · EmailTemplate · EmailTemplateVariable
Traçabilité	AuditLog

5.2 La contrainte EXCLUDE, ligne par ligne

```
ALTER TABLE bookings ADD CONSTRAINT ex_bookings_no_overlap
EXCLUDE USING gist (
    room_id WITH =,
    time_range WITH &&
)
WHERE (status <> 'annulee' AND deleted_at IS NULL);
```

Élément	Signification
EXCLUDE	Interdit qu'une paire de lignes satisfasse simultanément tous les opérateurs listés. Ce n'est pas une vérification : c'est une impossibilité.
USING gist	Index généralisé, seul capable d'indexer des intervalles. Nécessite l'extension <code>btree_gist</code> pour mêler un UUID (égalité) et un intervalle (chevauchement) dans le même index.
room_id WITH =	Deux lignes ne sont comparées que si elles concernent la même salle.
time_range WITH &&	<code>&&</code> est l'opérateur « se chevauche » de PostgreSQL sur les types intervalle.
WHERE (...)	Contrainte partielle : une réservation annulée ou supprimée ne bloque plus le créneau. Sans cette clause, annuler ne libérerait rien.

`time_range` est de type `TSTZRANGE`, en bornes `[début, fin)`. C'est ce qui règle proprement le cas adjacent : 10 h – 11 h et 11 h – 12 h ne se chevauchent pas, `&&` répond faux, la contrainte laisse passer.

5.3 Les autres garanties

Contrainte	Ce qu'elle rend impossible
uq_admin_accounts_single_owner	Deux comptes propriétaires.
uq_booking_access_codes_active	Deux codes valables pour une même porte.
uq_booking_participants_organizer	Deux organisateurs sur une réservation.
uq_booking_participants_email	Deux fois la même adresse invitée.
uq_booking_rules_global / _building / _room	Deux jeux de règles pour la même portée.
ck_bookings_cancelled_not_checked_in	Annuler une présence déjà validée.
ck_booking_access_codes_expiry	Un code émis après sa propre expiration.
ck_access_requests_reference_format	Une référence hors format <code>#ABC-1234</code> .

Les **51 index partiels** suivent la même logique : indexer seulement les lignes utiles — les réservations non supprimées, les codes non révoqués, les invitations en attente. L'index reste petit, donc rapide.

6. L'API — 160 opérations

128 chemins, quinze domaines, des conventions tenues partout.

Domaine	Opérations	Contenu
parc	33	Bâtiments, étages, salles, équipements, plans, photos, placements
comptes	21	Profils, préférences, sessions, comptes d'administration, permissions, invitations
support	20	Tickets, messages, gabarits, base de connaissances
réservations	15	Cycle complet, participants, codes, check-in, retard, alternatives
statistiques	12	Occupation, absences, rapports, export
règles	11	Règles par portée, prévisualisation, ouvertures, fermetures
authentification	10	Connexion, rafraîchissement, Google, mot de passe oublié
notifications	10	Liste, lecture, préférences, gabarits
Assistant	8	Conversations, messages, visites guidées
demandes d'accès	5	Ouverture, arbitrage, alternatives
administration	5	Tableau de bord, alertes
audit	4	Journal, détail, signalement, export CSV
disponibilité	4	Calendrier, vérification d'un créneau
recommandation	4	Classement des salles pour un besoin
technique	2	Santé, version

Conventions tenues sur les 160

- Les listes rendent `{ items, total, pagination }` : le total ne se devine pas depuis la page reçue.
- Les énumérations circulent en minuscules anglaises ; la traduction est affaire d'affichage.
- Les dates sont en ISO 8601 UTC, sans exception.
- Une entrée malformée rend **422 avec un code nommé**, jamais 500.
- Les erreurs suivent une forme unique : `{ error: { code, message } }`.
- Aucune route n'accepte un `owner_id` en paramètre : l'identité vient du jeton.

Authentification

Deux périmètres. Le jeton utilisateur porte `scope=user` ; celui de l'administration `scope=admin`, émis par `/auth/admin/login`. Un compte sans droits reçoit **le même refus qu'un mot de passe faux** — sans quoi la route dirait qui est administrateur.

Les permissions ne sont pas déduites du jeton : elles sont relues à chaque reprise de session. C'est le seul moyen qu'une révocation prenne effet avant la prochaine connexion.

7. Le front

52 écrans, 152 composants, 17 hooks, et un seul endroit qui connaît le réseau.

7.1 Organisation

src/	
api/	le seul endroit qui connaît HTTP
client.js	jeton en mémoire, rotation, enveloppe d'erreur
adapters.js	traduit les charges du serveur en objets d'écran
bookings.js	rooms.js checkin.js ... un module par domaine
admin/	les modules réservés à l'administration
hooks/	17 hooks
components/	152 composants, par domaine
pages/	52 écrans
utils/	format, dates, cadrage, partage
e2e/	parcours Playwright

7.2 Les hooks

Hook	Rôle
useAsync	Charge une ressource, expose <code>status</code> , <code>data</code> , <code>error</code> , <code>reload</code> . Utilisé par presque tous les écrans.
useAuth · useAdminSession	Les deux sessions. Le jeton vit en mémoire ; la continuité repose sur un cookie <code>httpOnly</code> .
useDataTable	Tri, pagination, sélection multiple pour les tableaux d'administration.
useFocusTrap	Piège le focus dans une modale, rend le focus en sortie, gère Échap.
useAvailability · useSlotCheck	Interrogent la disponibilité pendant le tunnel de réservation.
usePermission	Masque ce que le serveur refuserait. Ne remplace pas le contrôle serveur.
useMediaQuery · useInView · useSurfaceBloquante	Adaptation à la taille d'écran, animations à l'entrée, effacement des panneaux flottants devant une modale.
useToast · useDocumentTitle · useDebouncedValue · useBooking · useBookingActions · useSelectFilters	Notifications, titre de page, saisie différée, brouillon de réservation, actions, filtres.

7.3 Les écrans

Espace utilisateur — 33	Espace d'administration — 19
Accueil · Tunnel de réservation (4 étapes) · Mes réservations (liste et calendrier) · Détail · Modification · Validation de présence · Explorer les salles · Détail d'une salle · Plan du bâtiment · Notifications · Profil · Statistiques · Centre d'aide · Recherche globale · Accueil embarqué · Pages publiques	Tableau de bord · Statistiques et rapports · Toutes les réservations · Conflits et demandes · Bâtiments · Salles · Détail d'une salle · Équipements · Plans · Calendriers d'ouverture · Règles · Utilisateurs · Rôles et permissions · Tickets · Base de connaissances · Modèles d'e-mails · Journal d'audit · Profil

Une seule page, jamais deux versions

Aucun écran n'a de variante mobile séparée. Les tableaux denses deviennent des cartes sous 1024 px, les grilles se replient, les cibles tactiles passent à 44 px — mais c'est le même composant. Deux versions divergent toujours : l'une reçoit une correction que l'autre attend.

8. Les tests — 1 481

Fichier par fichier, avec ce que chacun défend.

8.1 Domaine — 182 tests, sans base

Fichier	Tests	Ce qu'il défend
domain/test_recommendation.py	43	Les six composantes, la sous-capacité éliminatoire, la justification générée.
domain/test_conflicts.py	36	Les six qualifications, le cas adjacent, les alternatives, l'arbitrage.
domain/test_rules.py	33	Les dix règles, une par une, et leur composition par <code>evaluate</code> .
domain/test_availability.py	30	Fusion, soustraction, fenêtres d'ouverture, changement d'heure.
domain/test_types.py	22	L'algèbre de <code>TimeSlot</code> : intersection, écart, extension, bornes.
domain/test_ponderation.py	18	La somme des poids vaut 100, et la classe refuse toute autre valeur.

8.2 Services et API — 511 tests, sur une vraie base

Fichier	Tests	Ce qu'il défend
services/test_parc.py	44	Bâtiments, étages, salles, équipements, plans.
services/test_api_v1.py	36	Conventions transversales : enveloppes, codes d'erreur, formats.
services/test_support.py	33	Tickets, messages, gabarits, base de connaissances.
services/test_administration.py	32	Comptes d'administration, invitations, gabarits d'e-mail.
services/test_reservations.py	31	Le cycle complet : création, modification, annulation, participants.
services/test_services.py	31	Disponibilité, règles, recommandation branchés sur la base.
services/test_autorisations.py	27	Chaque route protégée refuse ce qu'elle doit refuser.
services/test_auth.py	24	Connexion, rotation des jetons, réinitialisation.
services/test_messages.py	23	Notifications et courriels.
services/test_conflits_bornes.py	22	Les cas limites temporels : bornes, adjacence, changement d'heure.
services/test_google.py	19	Signature, émetteur, destinataire , adresse confirmée, compte suspendu.
services/test_comptes.py	19	Profils, préférences, sessions.
services/test_batiments.py	17	Hiérarchie bâtiment → étage → salle.
services/test_courriels.py	16	Rendu des gabarits, variables manquantes, file différée.
services/test_code_et_retard.py	14	Format du code, retard annoncé, annulation après validation.
services/test_code_acces.py	13	Réémission, révocation de l'ancien, salle sans badge.
services/test_profil.py	13	Modification du profil, photo, préférences.
services/test_concurrency.py	10	Deux transactions simultanées sur le même créneau.
services/test_tracabilite.py	8	Journal d'audit : contenu, diff, signalement.

services/test_organisation.py	8	Adresse interne ou externe, sur les deux schémas de comptes.
services/test_secret_notifications.py	7	Le code d'accès n'atteint jamais la base par la notification.
services/test_filtres_salles.py	5	Filtres bâtiments, étages, accessibilité — listes multiples.
services/test_detail_participants.py	4	Le détail porte ses participants, la liste ne les porte pas.
services/test_demarrage.py	2	L'application refuse de démarrer avec un secret d'usine.

8.3 Assistant — 144 tests

Fichier	Tests	Ce qu'il défend
ai/test_outils_assistance.py	27	Les outils que l'assistant peut appeler.
ai/test_garde_fous.py	22	Aucune écriture déclenchée par une sortie de modèle.
ai/test_outils.py	21	Contrat des outils : entrées validées, sorties bornées.
ai/test_fournisseurs.py	19	Bascule entre fournisseurs, repli si indisponible.
ai/test_repli_et_rag.py	19	Recherche documentaire et repli sans modèle.
ai/test_api_chat.py	18	Les routes de conversation.
ai/test_boucle.py	18	La boucle d'appel d'outils, bornée en profondeur.

8.4 Interface — 455 tests

Fichier	Tests	Ce qu'il défend
api/modules.test.js	71	Chaque module d'API envoie les bons paramètres.
api/adapters.test.js	64	Traduction des charges serveur, y compris champs absents.
components/admin/ecrans-corriges.test.jsx	39	Les écrans d'administration corrigés au fil du projet.
components/bookings/partage.test.jsx	25	Le partage n'empote jamais le code d'accès.
components/account/profil.test.jsx	21	Les quatre onglets du profil.
pages/onsite/fenetre-validation.test.jsx	18	La fenêtre de validation suit la règle du serveur.
api/client.test.js admin/regressions.test.js	30	Rotation du jeton, enveloppe d'erreur, régressions d'administration.
components/public/demo.test.jsx	13	La démonstration vidéo de la page publique.
components/admin/pagination-mobile.test.jsx	12	Pagination sur téléphone.
utils/cadrage.test.js	10	Recadrage de la photo de profil.
et 31 autres fichiers	152	Navigation, calendrier, consignes, focus, champs, cibles tactiles, textes longs, matrice de permissions, diff d'audit...

8.5 Parcours Playwright — 16

Fichier	Tests	Parcours
e2e/parcours-complet.spec.js	10	De la connexion à la confirmation, dans un vrai navigateur.

e2e/vitrine-animations.spec.js	4	La page publique et ses animations.
e2e/parcours-admin.spec.js	2	Connexion d'administration et arbitrage.

8.6 Les trois familles, et pourquoi elles coexistent

Famille	Ce qu'elle seule peut prouver
Domaine	Les cas limites en masse, sans coût. 182 tests s'exécutent en moins d'une seconde.
Intégration	Ce qui vit dans le moteur : exclusion sur intervalle, index partiels, contraintes. Un double en mémoire les ignorerait, et les tests seraient verts sur un système cassé.
Source	Ce que jsdom ne mesure pas : cibles tactiles à 44 px, coupure des textes longs, <code>min-w-0</code> sur les grilles. Le défaut ne se lit qu'à l'écran.

9. Les défauts trouvés et corrigés

La campagne finale, écran par écran. Ils partagent tous une forme.

La forme commune

Dans presque tous les cas : **deux couches nommaient la même chose différemment**, et rien ne le signalait. Un champ absent d'une réponse JSON ne lève pas d'erreur — il est simplement absent. Aucun test n'était rouge, aucun avertissement n'apparaissait ; seul l'écran affichait faux.

Défaut	Cause exacte
Matrice des permissions 28 cases vides pour 15 droits accordés	<code>admin.permissions.includes(permission.id)</code> comparait une liste de <i>codes</i> à un <i>UUID</i> . Le clic envoyait ce même UUID à une route qui attend des codes : refusé. Trois endroits touchés, plus le formulaire d'invitation.
Journal d'audit le bloc avant/après jamais affiché	L'adaptateur produit <code>entry.before</code> ; le composant lisait <code>entry.diff.before</code> — un niveau qui n'existe pas. Les 91 entrées portaient pourtant leurs valeurs.
Filtres de salles 3 sur 5 sans effet	L'écran envoyait <code>buildings</code> , <code>floors</code> , <code>accessible</code> ; la fonction déstructurait <code>building</code> , <code>floor</code> , <code>accessibleOnly</code> . Le plus grave : le filtre accessibilité PMR.
Écran de modification plantage complet	<code>booking.participants.filter()</code> sur <code>undefined</code> : le détail ne portait pas les participants. La fiche affichait « Participants (0) » en silence.
Code d'accès dans les notifications	Un seul rendu servait le courriel et la notification. La seconde est écrite en base : le code y restait en clair. 23 notifications corrigées.
Validation de présence	L'écran annonçait une ouverture 10 min <i>avant</i> le début ; le serveur ouvre <i>au</i> début. Six 422 par tentative. Le chronomètre était figé et le tiret du code retiré avant l'envoi.
Cibles tactiles	« Voir », « Traiter », « Réinitialiser » à 16 px — sous le minimum de 24 px. Portées à 44 px au tactile, densité rendue au-delà de 1024 px.
Focus dans les modales	<code>useFocusTrap</code> dépendait de <code>onEscape</code> , recréé à chaque rendu : chaque lettre frappée renvoyait le focus sur la croix de fermeture.
Adresse d'un ticket	<code>/admin/tickets/:id</code> existait dans le routeur mais la page ne lisait jamais le paramètre.

Chaque correction est accompagnée du test qui échouait avant elle. C'est la seule preuve que le défaut existait, et la seule garantie qu'il ne reviendra pas.

10. Questions du jury

Les questions probables, et de quoi y répondre sans hésiter.

« Pourquoi une contrainte base plutôt qu'une vérification applicative ? »

Parce qu'une vérification applicative laisse une fenêtre entre le contrôle et l'écriture. Deux requêtes simultanées passent toutes deux le contrôle, et toutes deux écrivent. Le défaut n'apparaît qu'en charge — donc jamais pendant les tests manuels. La contrainte supprime la fenêtre : quel que soit le chemin d'écriture, y compris un script ou une migration, le chevauchement est impossible. Nous gardons la vérification applicative, mais pour *expliquer* le refus, pas pour le garantir.

« Comment sont gérés les créneaux adjacents ? »

Les intervalles sont fermés-ouverts : `[10h, 11h)` et `[11h, 12h)` ne se chevauchent pas, l'opérateur `&&` répond faux, la contrainte laisse passer. Le domaine reconnaît explicitement le cas `ADJACENT` et l'exclut de l'ensemble `RECouvreMENTS`. En revanche, le *battement* configuré — 15 minutes par défaut — peut rendre deux créneaux adjacents incompatibles : c'est une règle métier, pas une contrainte d'intégrité, et elle est donc vérifiée par `check_buffer`.

« Que se passe-t-il au changement d'heure ? »

Tout est stocké en UTC ; seule l'expression des horaires d'ouverture est locale. La conversion se fait **jour par jour** avec `ZoneInfo`, jamais par un décalage constant. Le dernier dimanche d'octobre compte 25 heures : une fenêtre « 8 h – 20 h » y dure 13 heures, pas 12. `test_conflicts_bornes.py` couvre ces cas.

« Comment sont pondérées les recommandations ? »

Six composantes : capacité 30, équipements 25, bâtiment 15, occupation 12, étage 10, historique 8 — somme 100, vérifiée à la construction de la classe qui lève si le total diffère. La sous-capacité est éliminatoire et non pénalisante. Le score ordonne mais n'élimine jamais : l'éligibilité est décidée avant, par la disponibilité et les règles. Chaque proposition porte sa justification, générée depuis les composantes.

« Comment la concurrence est-elle testée ? »

`test_concurrency.py` ouvre deux transactions sur le même créneau et vérifie qu'une seule aboutit, l'autre recevant la violation de contrainte traduite en `SlotConflictError`. Le test tourne sur une vraie base : c'est le seul moyen, la contrainte n'existant que dans PostgreSQL.

« Quelle est la répartition du travail dans l'équipe ? »

Réponse à préparer ensemble et à donner franchement. Le jury pose presque toujours cette question sur un projet dont l'ampleur dépasse le cadre du sujet. Dire qui a tenu quelle partie, et ce que chacun maîtrise réellement, passe bien mieux qu'une réponse vague. Si l'un de vous n'a pas écrit une partie, le dire et expliquer comment il l'a comprise.

« Quels arbitrages de périmètre avez-vous faits ? »

Trois notables. Nous n'avons pas ajouté de bibliothèque SAML pour l'authentification ADFS : la connexion Google remplit le même besoin. Nous n'avons pas fait de campagne de charge, et nous n'affichons donc aucun chiffre de performance. Nous avons choisi des occurrences **indépendantes** pour les réservations récurrentes, ce qui évite la question insoluble des exceptions et rend chaque occurrence arbitrage seule.

« Qu'est-ce qui est développé, et qu'est-ce qui est maqueté ? »

Tout ce qui est présenté est fonctionnel et branché sur l'API. L'assistant fonctionne sur un modèle local et n'a pas été confronté à de vrais usages — c'est la partie la moins éprouvée, et nous le disons.

« Pourquoi tester sur une vraie base ? »

Parce que l'essentiel des garanties vit dans le moteur : exclusion sur intervalle, 51 index partiels, 110 contraintes. Un double en mémoire les ignorerait, et les tests seraient verts sur un système cassé.

« **Quelle est la partie dont vous êtes le plus fiers ?** »

L'isolation du domaine. `app/domain/` contient des fonctions pures qu'on peut lire, tester et discuter sans dérouler une requête SQL. C'est ce qui a permis de corriger des règles en fin de projet sans rien casser ailleurs.

« **Qu'est-ce qui vous a le plus surpris ?** »

Que les défauts les plus graves soient les plus silencieux. Un écran qui affiche zéro permission alors que la base en porte quinze ne lève aucune erreur : il faut le *regarder* pour s'en apercevoir. C'est ce qui nous a conduits à écrire des tests qui lisent le code source lui-même.